🚀 DSA Mastery Guide (2026 Edition)

The 8 Core Patterns to Crack 80–90% Coding Interviews

🎯 Who is this for?

- SDE 1 / SDE 2 candidates
- FAANG / MAANG aspirants
- Anyone stuck in **random LeetCode practice**

🧠 Why These 8 Patterns Matter

Most candidates fail because:

- They solve problems randomly
- They don't recognize patterns

👉 Interviewers don't test questions.
👉 They test **patterns + thinking**.

🔥 If you master these 8 patterns:

You can solve:

- 70–80% LeetCode problems
- Most interview questions

🟢 1. TWO POINTERS

@ts.unfiltered.aman

🎯 Core Idea

Use **two indices** to traverse data efficiently.

🧠 When to Use

- Sorted arrays
- Opposite ends traversal
- Removing duplicates
- Partitioning

💡 Variations

- Left → Right
- Fast & Slow pointer
- Converging pointers

🧪 Common Problems

- Two Sum (sorted)
- Move Zeros
- Container With Most Water
- Remove Duplicates

⚡ Example Thinking

Instead of O(n²), use:
👉 Two pointers → O(n)

🔥 Pro Tip

@its.unfiltered.aman

👉 Always ask:
“Can I reduce nested loops using two pointers?”

🟡 2. SLIDING WINDOW

🎯 Core Idea

Process **subarrays / substrings** using a moving window.

🧠 When to Use

- Contiguous subarrays
- Maximum / minimum window
- Longest substring problems

💡 Types

- Fixed window
- Dynamic window

🧪 Common Problems

- Maximum Average Subarray
- Longest Substring Without Repeating Characters
- Minimum Window Substring

⚡ Example Thinking

@its.unfiltered.aman

Instead of recalculating:
👉 Expand → Shrink → Optimize

🔥 Pro Tip

👉 Use when question says:
“subarray” / “substring”

🔵 3. MODIFIED BINARY SEARCH

🎯 Core Idea

Apply binary search on **answer space**, not just arrays.

🧠 When to Use

- Sorted data
- Optimization problems
- “Minimum/Maximum possible value”

💡 Key Trick

👉 Convert problem into:
“Can this answer work?”

🧪 Common Problems

- Koko Eating Bananas
- Capacity to Ship Packages

@its.unfiltered.aman

- Find Peak Element

⚡ Example Thinking

👉 Instead of searching value
Search answer range (low → high)

🔥 Pro Tip

👉 Look for:
“minimum possible”, “maximum feasible”

🟣 4. HASHMAP / HASHSET

🎯 Core Idea

Trade **space for speed** using fast lookups.

🧠 When to Use

- Frequency counting
- Lookup problems
- Avoid nested loops

💡 Operations

- Insert → O(1)
- Lookup → O(1)

@its.unfiltered.aman

🧪 Common Problems

- Two Sum
- Majority Element
- Group Anagrams
- Contains Duplicate

⚡ Example Thinking

👉 Replace O(n²) with hashmap → O(n)

🔥 Pro Tip

👉 First thought in interviews:
“Can I use a hashmap?”

🟠 5. PREFIX SUM / MONOTONIC STACK

🎯 Core Idea

Precompute data to answer queries instantly.

🧠 When to Use

- Range sum queries
- Subarray problems
- Increasing/decreasing patterns

@its.unfiltered.aman

💡 Concepts

Prefix Sum:

- Precompute cumulative sum

Monotonic Stack:

- Maintain sorted stack

🧪 Common Problems

- Subarray Sum Equals K
- Daily Temperatures
- Largest Rectangle in Histogram

⚡ Example Thinking

👉 Precompute once → Answer fast

🔥 Pro Tip

👉 If problem asks:
“sum of subarray” → think prefix sum

🔴 6. BFS / DFS

🎯 Core Idea

Traverse graphs/trees.

@its.unfiltered.aman

🧠 When to Use

- Trees
- Graphs
- Matrix traversal

💡 Types

BFS (Queue)

- Level-by-level
- Shortest path

DFS (Stack/Recursion)

- Explore deeply
- Backtracking

🧪 Common Problems

- Number of Islands
- Word Ladder
- Binary Tree Traversal

⚡ Example Thinking

👉 Grid → DFS/BFS
👉 Shortest path → BFS

🔥 Pro Tip

👉 Anytime you see:

@its.unfiltered.aman

- Grid
- Graph
- Tree

👉 Think BFS/DFS immediately

🟡 7. MEMOIZATION (DP)

🎯 Core Idea

Avoid recomputation using caching.

🧠 When to Use

- Overlapping subproblems
- Recursive solutions
- Optimization problems

💡 Approach

- Top-down (memoization)
- Bottom-up (DP table)

🧪 Common Problems

- Climbing Stairs
- Coin Change
- House Robber

@its_unfiltered.aman

⚡ Example Thinking

👉 If recursion repeats → use memoization

🔥 Pro Tip

👉 Ask:
“Am I solving the same problem again?”

⚫ 8. MIN / MAX HEAP

🎯 Core Idea

Efficiently track smallest/largest elements.

🧠 When to Use

- Top K problems
- Priority-based problems
- Streaming data

💡 Structure

- Min Heap
- Max Heap

🧪 Common Problems

@its.unfiltered.aman

- Top K Frequent Elements
- Last Stone Weight
- Merge K Sorted Lists

⚡ Example Thinking

👉 Maintain top K → Heap

🔥 Pro Tip

👉 Look for:
“Top K”, “smallest”, “largest”

🧠 FINAL REVISION TABLE

Pattern	When to Use
Two Pointers	Sorted arrays, pair problems
Sliding Window	Subarray/substring
Binary Search	Optimization problems
Hashmap	Fast lookup
Prefix Sum	Range queries
BFS/DFS	Graphs/trees
Memoization	Repeated recursion
Heap	Top K problems

🚀 HOW TO MASTER THESE

@its.unfiltered.aman

🟢 Step 1

Pick 1 pattern per day

🟢 Step 2

Solve 5–10 problems

🟢 Step 3

Focus on:

- Pattern recognition
- Not memorization

🟢 Step 4

Revise weekly

⚠️ COMMON MISTAKES

❌ Solving random problems
❌ Ignoring patterns
❌ Not revising
❌ Memorizing solutions

@ig.unfiltered.aman

🏁 FINAL ADVICE

👉 Interviews are not about solving 1000 problems

👉 They are about recognizing patterns quickly

💬 Final Thought

Master patterns → Not problems → Crack interviews

@its.unfiltered.aman